

NexusAI

TECHNICAL AUDIT & PRODUCT STRATEGY

An honest engineering review of the codebase, a competitive positioning analysis, and a prioritized roadmap to make the platform genuinely production-grade.

SCOPE REVIEWED

React 19 + Vite frontend (40+ feature pages) • Node/Express + Socket.IO backend
MongoDB/Mongoose • Broker adapter layer • Paper-trading engine • Razorpay
Gemini AI copilot • Solidity smart contracts (Polygon)

Prepared by an external code review • Confidential

1. EXECUTIVE SUMMARY

NexusAI is one of the most ambitious student / portfolio projects you can build in this space. The scope is large and largely real: a centralized live market-data engine, a paper-trading execution engine with risk guards, a broker abstraction layer with four adapters, WebSocket hardening, security middleware, a Razorpay wallet, an AI copilot, and on-chain prediction logging. The engineering instincts on display — singletons as a single source of truth, exponential backoff, health endpoints, idempotency, async DB flushing — are well above typical.

However, "ambitious and broad" is not the same as "production-grade." The platform currently has a thin but critical layer of problems that an experienced reviewer (or interviewer) will find in minutes: authentication is effectively a mock, the premium paywall is bypassed in code, an idempotency check can never fire because of a timestamp bug, secrets are committed, and there are no automated tests. None of these are hard to fix — and fixing them is exactly what converts this from "impressive demo" to "credible product."

Overall verdict

Scope & ambition

Exceptional — breadth rivals real fintech MVPs.

Engineering craft

Strong — patterns are mature; comments show intent.

Production readiness

Not ready — auth, secrets, and tests are blocking.

Differentiation

Promising but unfocused — too many half-features.

The one-line recommendation

Stop adding features. Spend the next phase hardening what exists — real authentication, a real paywall, tests, and honest "simulated data" labelling — then pick ONE differentiator (institutional-grade risk analytics, or the on-chain verifiable track record) and make it genuinely excellent. A focused, trustworthy platform beats a 40-page one that cannot be trusted with a login.

2. WHAT IS GENUINELY STRONG

These are real strengths, grounded in the code — lead with them when you present or interview.

1. A true single source of truth for pricing

MarketDataService is a singleton that every consumer (charts, OMS, portfolio, simulator) reads from. It batches Yahoo Finance quotes, tracks provider health, applies exponential backoff on HTTP 429, falls back to last-known-good prices, and reduces poll frequency outside market hours. This is the correct architecture and it directly solved a real bug (chart price "`` execution price).

```
server/services/MarketDataService.js – backoff, fallback cache, isMarketOpen()
```

2. A paper-trading engine that thinks like an exchange

PaperTradingEngine executes in-memory for low latency, then flushes to MongoDB asynchronously every 5s. It models slippage, enforces a max-position-size limit (25% of capital), rejects insufficient-funds and short-sells, has a kill-switch, and emits risk events. That risk-first mindset is exactly what distinguishes a trading system from a CRUD app.

```
server/services/PaperTradingEngine.js:60-163 – execution + risk guards
```

3. Broker abstraction done properly

BrokerGateway routes to Zerodha / Upstox / Angel One / Mock adapters behind one interface, degrades to a sandbox adapter when credentials are missing, computes per-broker health scores, and keeps a telemetry buffer. The frontend stays broker-agnostic. This is a clean, extensible pattern.

```
server/services/BrokerGateway.js – adapter map + getDiagnostics()
```

4. Operational maturity

- % Deep health checks: /api/health/deep reports DB, market-data age, WS clients, and memory.
- % WebSocket hardening: reconnection-flood throttling, room-based subscriptions, stale-socket cleanup.
- % Security middleware stack: helmet, mongo-sanitize, tiered rate limiters (auth/wallet/trade).
- % Graceful degradation everywhere: fallbacks for prices, history (Brownian bridge), and broker failures.

3. CRITICAL ISSUES FOUND IN THE CODE

These are concrete, file-level findings. They are ordered by severity. Fix the P0 items before showing this to anyone evaluating it as a product.

P0 — BLOCKING

3.1 Authentication is a mock — anyone is "logged in"

The /login route ignores the password entirely and returns a static user id. The /me route then reports kyc_status: "VERIFIED" and isPremium: true for that mock user. There is no password hashing, no session, no real identity for Web2 users.

```
server/routes/auth.js:15-21 - login returns { user_id: "mock_web2_user" } with no check
server/routes/auth.js:43-50 - hard-codes VERIFIED / isPremium:true
```

%, Fix: implement real email+password (bcrypt) or commit fully to wallet-only auth (you already have a solid nonce/SIWE flow in /nonce + /verify). Issue a JWT and verify it on every protected route.

P0 — BLOCKING

3.2 The premium paywall is bypassed in middleware

requirePremium contains a line that flips every non-premium user to premium instead of rejecting them. Combined with dev-mode short-circuits for ids like "mock_web2_user" and "1", the entire paywall is non-functional. Razorpay can take money, but nothing actually gates on payment.

```
server/middleware/auth.js:53-56 - if (!user.isPremium) { user.isPremium = true; }
```

%, Fix: delete the auto-upgrade; return 403 when !user.isPremium. Remove the broad dev/test id allow-list from production builds (guard strictly on NODE_ENV).

P0 — SECURITY

3.3 Secrets are committed and weakly defaulted

%, docker-compose.yml hard-codes JWT_SECRET="nexusai_super_secret_production_key" in plaintext.

%, JWT_SECRET falls back to a known string in two files, so a missing env silently uses a guessable key.

%, server/.env is present in the tree — verify it is gitignored and rotate anything that leaked.

```
docker-compose.yml:12 • server/middleware/auth.js:4 • server/routes/auth.js:6
```

%, Fix: move all secrets to env/secret manager, fail fast (exit) if JWT_SECRET is unset, and scrub git history if real keys were ever committed.

P1 — CORRECTNESS

3.4 Order idempotency can never trigger

The idempotency key embeds Date.now(), so every order produces a unique key. The duplicate-detection set therefore never matches, and the cache is cleared 5s later anyway. The safety net is effectively dead code — a double-click or client retry will place two orders.

```
server/services/BrokerGateway.js:156 - key = `${...}_${Date.now()}`
```

%, Fix: have the client send a stable Idempotency-Key (UUID per user action); key the cache on that, not on time.

P1 — RELIABILITY

3.5 In-memory execution risks silent data loss

Balances and positions mutate in memory and flush every 5s. A crash between flushes loses trades; the code comment acknowledges this. There is also no per-user lock around the async loadUserContext !' mutate sequence, so interleaved orders can read a stale balance.

server/services/PaperTradingEngine.js:178-218 — flushToDB, no retry/back-pressure

% Fix: write-ahead the order before mutating, use atomic \$inc on the account doc as the source of truth, and re-queue failed flushes instead of dropping them.

P2 — TRUST

3.6 Simulated data is presented as live market signal

Microstructure order books, "Block Deal Detected" smart alerts, and the slippage model are generated from counters and Math.random(). For a demo that is fine — but unlabelled fake signals in a finance product erode trust fast and would be a compliance problem in production.

server/index.js:447-496 — microTick order book + cycling alerts

% Fix: badge every synthetic feed as "Simulated / Demo" in the UI, and gate real signals behind a clearly separate data source.

P2 — QUALITY

3.7 No automated tests or CI

There are ad-hoc scripts (stress_test.js, e2e_test.js) but no test framework, no assertions, no CI pipeline. For a system handling money-like state, this is the highest-leverage gap after auth.

% Fix: add Vitest/Jest for the OMS and ledger math (the parts where bugs cost money), plus a GitHub Actions lint+test gate.

Other notes

% Singletons + setInterval timers mean two server instances would double-broadcast and double-poll — horizontal scaling needs Redis adapter + a leader for timers.

% Gemini JSON is parsed via regex with no schema validation or per-route rate limit (cost risk on the AI endpoint).

% Root package.json is still name:"frontend", version:"0.0.0" — polish before sharing.

% CORS production config allows any localhost origin — tighten for real deployment.

4. HOW TO BE BETTER THAN OTHER PLATFORMS

You cannot out-Zerodha Zerodha on execution, or out-TradingView TradingView on charts — they have years and capital. The winning move is not "more features," it is a sharp, defensible angle. Here is an honest read of the landscape and where NexusAI can actually win.

The landscape (be realistic)

Zerodha / Upstox / Angel One

Own real execution, licences, liquidity. You will not beat them there.

TradingView

Owns charting + social. Deep moat on visualization.

Groww / INDmoney

Own the simple, friendly retail onboarding experience.

Smallcase

Owns thematic/basket investing.

Where NexusAI can genuinely differentiate

- % Institutional-grade risk, retail-simple UX. Portfolio X-Ray, stress tests, correlation/beta, and "what happens to MY holdings in a 2008-style crash" is something the big brokers do NOT surface well. Own "understand your risk before you trade."
- % A verifiable, tamper-proof track record. Your on-chain PredictionLogger is a real differentiator: let users (or an AI) publish a prediction, hash it on Polygon, and prove later they called it — no screenshot fakery. "Provable alpha" is a story no broker tells.
- % A genuinely useful AI copilot — not a chatbot gimmick. Wire Gemini to the user's actual portfolio + live data and have it explain concentration risk, suggest hedges, and answer "why did my P&L move today." Grounded, personal, explainable.
- % A serious paper-trading / learning arena. Your engine already models slippage and risk limits. Lean into education: tournaments, a trading-DNA profile, post-trade analytics — the best risk-free place to learn Indian markets.

What to cut or de-emphasize

Forty feature pages dilute the story and multiply the surface area of half-working code. Pick the 3-4 that ladder up to your chosen angle (e.g. Portfolio X-Ray, Stress Test, AI Copilot, On-chain track record) and make them excellent. Move the rest behind a "Labs" flag or remove them.

5. PRIORITIZED ROADMAP

A sequenced plan. Do not skip ahead — the P0 layer is what makes everything above it believable.

PHASE 0 — TRUST (1-2 weeks)

- % Real auth: bcrypt email/password OR full wallet-SIWE; verify JWT on every protected route.
- % Fix the paywall: remove the auto-premium line; return 403; strict NODE_ENV guards.
- % Secrets: env/secret manager, fail-fast on missing JWT_SECRET, rotate + scrub history.
- % Fix idempotency with a client-supplied key; write-ahead orders before mutating balances.
- % Label every simulated feed clearly in the UI.

PHASE 1 — CONFIDENCE (2-3 weeks)

- % Tests: Vitest on OMS + ledger math; e2e happy-path; GitHub Actions lint+test gate.
- % Observability: structured logs already exist — add error tracking (Sentry) and a status page.
- % Harden the AI route: schema-validate Gemini output, add a per-user rate limit and a cost cap.
- % Pick the ONE differentiator and write the product narrative around it.

PHASE 2 — SCALE & POLISH (3-6 weeks)

- % Redis: Socket.IO adapter for multi-instance fan-out; move shared state out of singletons; elect a single timer leader.
- % Real data source for the chosen differentiator (or a clearly-licensed market feed).
- % Deepen the on-chain track record: verification UI, shareable proof links.
- % Accessibility + mobile pass on the core 4 pages; performance budget on the dashboard.

Highest-leverage quick wins (do this week)

- % Delete auth.js:53-56 auto-premium line.
- % Remove the plaintext JWT_SECRET from docker-compose.yml.
- % Replace Date.now() in the idempotency key.
- % Add a "Simulated" badge component and drop it on the microstructure + alerts panels.
- % Write 10 unit tests for the buy/sell ledger math.

6. HOW TO PRESENT THIS (RESUME & PITCH)

The project is a strong signal IF you frame it with engineering honesty. Interviewers and reviewers respect "I built X, found these flaws, and here is how I fixed them" far more than "everything works perfectly."

Resume bullets (accurate, quantified)

- % Built a real-time trading platform: centralized market-data singleton (Yahoo Finance) feeding a Socket.IO broadcast layer with reconnection throttling and room-based subscriptions.
- % Designed a paper-trading execution engine with in-memory fills, slippage modelling, position-size limits, and async MongoDB persistence.
- % Implemented a broker abstraction layer (4 adapters) with health scoring and automatic sandbox fallback.
- % Added an on-chain (Polygon/Solidity) prediction logger for tamper-proof, verifiable forecasts.

The honesty that wins interviews

Be ready to say: "Auth started as a mock so I could iterate on the trading core; my next step is real SIWE/JWT. I also found that my idempotency key included a timestamp, so it never deduplicated — I fixed it with a client-supplied key. The slippage and microstructure feeds are simulated and labelled as such." This turns every flaw in this report into evidence that you can audit and harden your own systems — which is exactly what senior engineering is.